1.4. Particularidades de la programación de guiones (scripts) y sus ventajas y desventajas sobre la programación tradicional (Criterio 1.4)

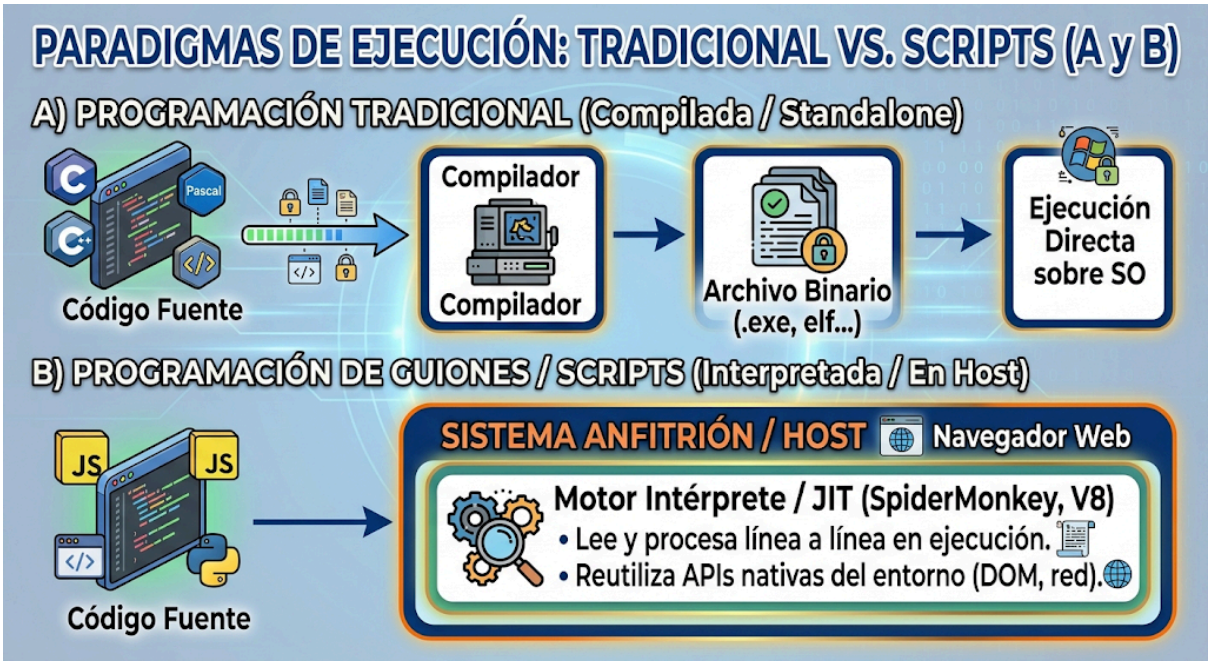
- **Resultado de Aprendizaje Asociado: RA1** (*Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una*).

- **Criterio Curricular Oficial: CE 1.d** (Se han reconocido las particularidades de la programación de guiones y sus ventajas y desventajas sobre la programación tradicional).

- **Ponderación del Criterio:** (**16,67% del RA1** | **0,833% sobre la calificación final del módulo**).

El desarrollo de software ha experimentado una profunda evolución conceptual. Tradicionalmente, los lenguajes de programación se concebían para construir aplicaciones aisladas e independientes (*standalone*) destinadas a la gestión empresarial clásica (nóminas, contabilidad, procesadores de texto, hojas de cálculo o almacenes) sobre sistemas operativos de escritorio o arquitecturas cliente/servidor monolíticas.

La llegada de internet, la ubicuidad de los dispositivos móviles (teléfonos inteligentes y tabletas) y la necesidad de ejecutar lógica dentro de navegadores web transformaron los requisitos técnicos. Ya no se programa todo desde cero: se emplean entornos anfitriones (*host systems*) para acoplar y ejecutar rutinas dinámicas.



A. Origen, Concepto y Naturaleza de los Scripts

- **Nacimiento de los scripts:** Surgieron como secuencias de comandos o pequeños fragmentos de código diseñados para automatizar tareas rutinarias y repetitivas en los sistemas operativos.
- **Dependencia del intérprete:** Los scripts siempre son ejecutados por un intérprete de comandos o motor de ejecución subyacente.
- **De pequeñas macros a programas complejos:** En la actualidad, los scripts han superado su concepción como simples rutinas auxiliares. En la web actual representan programas completos con arquitecturas complejas de miles de líneas de código, manejando estados, interfaces reactivas y comunicaciones de red.

B. Diferencias Fundamentales entre Lenguajes de Script y Lenguajes Tradicionales

El contraste entre ambos mundos define sus posibilidades técnicas y operativas:

- **1. Proceso de compilación frente a interpretación:**
    - *Lenguajes tradicionales:* Requieren un paso previo de compilación que traduce el código fuente a código máquina binario específico para una plataforma. Sin esa fase de compilación, el programa no existe como ejecutable.
    - *Lenguajes de script:* Son interpretados directamente. El motor procesa y evalúa las instrucciones línea a línea en tiempo de ejecución, sin requerir una compilación previa por parte del programador ni generar un archivo ejecutable intermedio independiente.
- **2. Ejecución independiente frente a integración en un sistema anfitrión (*host*):**
    - *Lenguajes tradicionales:* Crean programas *standalone (programas independientes que se ejecutan directamente en un sistema operativo sin necesidad de instalar un entorno de desarrollo).* **Compilados Nativos (C++, Go, Rust):** Crean binarios autónomos (.exe) directamente.
    - **Gestionados por Entorno/Máquina Virtual (Java, C#):** Se compilan a un código intermedio y requieren un entorno instalado (JVM / .NET) para ejecutarse, aunque hoy permiten empaquetarse como *standalone*.
    - **Interpretados / Script (Python, JavaScript):** Leen el código fuente línea por línea mediante un intérprete, requiriendo su propio entorno o un navegador (host), pero también soportan empaquetado moderno.

      Aunque nacieron para requerir un sistema anfitrión contenedor (como JavaScript dentro de un documento HTML en el navegador), hoy en día pueden ejecutarse de manera autónoma.

        - **En consola:** Python y JavaScript (mediante Node.js) pueden ejecutarse directamente sobre el sistema operativo desde la terminal.
        - **Como Standalone:** Ambos lenguajes permiten empaquetar el código junto con su intérprete utilizando herramientas externas (como *PyInstaller* para Python o *pkg/Electron* para JavaScript), transformándolos también en aplicaciones independientes y ejecutables.

- **3. Desarrollo desde cero frente a reutilización de componentes preexistentes:**
    - *Lenguajes tradicionales:* Construyen a menudo sus propias estructuras, interfaces y librerías desde la base.
    - *Lenguajes de script:* Nacieron diseñados para apoyarse y enlazar componentes que ya existen en el sistema anfitrión (como los elementos del DOM, el motor gráfico o las llamadas de red del navegador, en el caso de JavaScript,).
- **4. Momento de detección de errores:**
    - *Lenguajes tradicionales:* La fase de compilación actúa como un filtro estricto de sintaxis y tipos; si existe un fallo estructural, el binario ejecutable no llega a generarse.
    - *Lenguajes de script:* Al ejecutarse línea a línea directamente en el entorno del cliente, los fallos sintácticos o de asignación se descubren durante el tiempo de ejecución (*runtime*), lo que exige planes de prueba exhaustivos.
- **5. Clasificación de lenguajes:**
    - *Lenguajes de programación tradicional:* C, C++, Java, Swift, Pascal.
    - *Lenguajes de scripting:* JavaScript, Shell script, Perl, PHP, Python, Ruby.

C. Análisis de Ventajas y Desventajas de la Programación de Guiones

Ventajas Destacadas

- **Sencillez y curva de aprendizaje rápida:** Han sido diseñados específicamente para que sean fáciles de utilizar y programar, reduciendo la complejidad formal de los lenguajes tradicionales.
- **Agilidad en el ciclo de desarrollo:** No requieren esperar tiempos de compilación ni enlazar binarios. Cualquier cambio en el código fuente se comprueba al instante recargando la página en el navegador.
- **Integración natural:** Facilidad absoluta para incrustarse dentro de otros lenguajes o documentos, como ocurre con la integración directa de JavaScript dentro de las etiquetas de un archivo HTML.
- **Portabilidad mediante el anfitrión:** El código JavaScript puede funcionar de manera multiplataforma en cualquier ordenador, tableta o smartphone siempre que disponga de un navegador compatible con los estándares.

Desventajas y Riesgos Técnicos

- **Mayor tasa de errores en tiempo de ejecución:** Al interpretarse en caliente, un fallo en una rama de código condicional poco transitada puede pasar desapercibido hasta que el usuario final interactúa con ese elemento concreto.
- **Rendimiento bruto inferior:** Aunque los motores modernos aplican compilación en tiempo real (*JIT*), un lenguaje interpretado dinámico consume más memoria y ciclos de procesador que un ejecutable binario en C o C++ optimizado.
- **Exposición del código fuente:** En el caso de JavaScript, al transferirse al cliente como texto plano para ser interpretado en su navegador, el código queda expuesto de forma pública ante cualquier usuario.

D. Casos Singulares y Proyección del Ecosistema de Scripting

- **La proyección de Python en el mercado:** Dentro de los lenguajes de scripting, Python destaca por tener una proyección enorme al ser el lenguaje de referencia en

el desarrollo de inteligencia artificial, computación científica y tratamiento masivo de datos.

- **El caso de Java vs. JavaScript:** A pesar de la similitud en sus nombres por razones comerciales en su origen histórico, son lenguajes con filosofías opuestas:
    - *Java:* Lenguaje de programación tradicional, fuertemente tipado, compilado a *bytecode*, orientado rígidamente a objetos y ejecutable en una máquina virtual.
    - *JavaScript:* Lenguaje de script, dinámico, débilmente tipado, interpretado directamente en el navegador y orientado a eventos.

E. El Objeto Date en JavaScript

1. Naturaleza y Modelo Interno de las Fechas

En JavaScript, las fechas no son un tipo de dato primitivo, sino instancias del objeto nativo Date.

- **Representación temporal fija:** Un objeto Date contiene una instantánea congelada en el tiempo (un punto estático). No se actualiza dinámicamente como un reloj en tiempo real.
- **Época Unix (*Epoch Time*):** Internamente, JavaScript almacena la fecha como un número entero: el número de **milisegundos transcurridos desde el 1 de enero de 1970 a las 00:00:00 UTC** (*Coordinated Universal Time*).
    - Un valor positivo indica instantes posteriores a esa fecha.
    - Un valor negativo indica instantes anteriores a 1970.
    - $1día = 24 \times 60 \times 60 \times 1000 = 86.400.000ms$.

```
// Obtención del timestamp Unix actual en milisegundos sin instanciar un objeto
const tiempoActualMs = Date.now();
console.log(tiempoActualMs); // Ejemplo: 1790591037183
```

2. Formas de Instanciación (new Date)

Existen cuatro variantes principales del constructor:

A. Sin argumentos (Fecha y hora actual)

Captura el instante exacto según el reloj del sistema local:

```
const ahora = new Date();
```

B. Mediante cadena de texto (*Date String*)

Interpreta fechas expresadas en formatos estándar reconocidos por el analizador sintáctico (ISO 8601 o RFC 2822):

```
// Formato recomendado ISO 8601 (YYYY-MM-DD o YYYY-MM-DDTHH:mm:ss)
const fechaISO = new Date("2026-09-28");
const fechaHora = new Date("2026-09-28T12:30:00");
```

C. Por componentes numéricos (año, mes, [día, horas, minutos, segundos, ms])

Permite pasar entre 2 y 7 parámetros enteros:

```
// new Date(año, mesIndex, día, hora, minutos, segundos, milisegundos)
const navidad = new Date(2026, 11, 25, 10, 30, 0, 0);
```

Regla de indexación de meses (0 a 11):

En JavaScript, los meses son de base cero (*zero-indexed*):

- 0 = Enero, 1 = Febrero, ..., 11 = Diciembre.
- Los días del mes (1 a 31), en cambio, van del 1 en adelante.

Comportamiento de desbordamiento (*Overflow* automático)

Si se asignan valores superiores a los límites naturales del mes o del día, el motor calcula el exceso y avanza automáticamente a la siguiente unidad temporal:

```
// Mes 15 -> 2026 + 1 año (12 meses) + 3 meses restantes = Abril de 2027
const fechaMesExcedido = new Date(2026, 15, 20);

// Día 35 en junio (junio tiene 30 días) -> 5 de julio
const fechaDiaExcedido = new Date(2026, 5, 35);
```

Tratamiento de años con uno o dos dígitos

Si el primer argumento está entre 0 y 99, JavaScript asume que corresponde al siglo XX (1900–1999):

```
const fechaPasada = new Date(95, 5, 15); // 15 de junio de 1995
```

D. Mediante milisegundos desde la época Unix

Si se pasa un **único número** entero al constructor, se interpreta siempre como milisegundos transcurridos desde el 1 de enero de 1970:

```
const inicioUnix = new Date(0); // Thu Jan 01 1970 01:00:00 GMT+0100 (hora local española)
const unDiaDespues = new Date(86400000); // 2 de enero de 1970

// OJO: Pasar un solo número NUNCA indica el año
const errorComun = new Date(2026); // Interpreta 2026 milisegundos después de 1970
```

3. Métodos Principales de Conversión y Salida

Método	Estándar de Formato	Salida Típica	Caso de Uso

toString()	Texto completo con zona horaria local	Mon Sep 28 2026 12:23:57 GMT+0200 (CEST)	Depuración rápida / Conversión por defecto
toDateString()	Solo fecha en formato legible	Mon Sep 28 2026	Interfaces sin detalle de horas
toTimeString()	Solo hora con huso horario	12:23:57 GMT+0200 (CEST)	Registros de eventos horarios
toISOString()	Estándar ISO 8601 en tiempo universal (UTC)	2026-09-28T10:23:57.000Z	**Intercambio de datos con APIs y bases de datos**
toUTCString()	Estándar HTTP / RFC 7231	Mon, 28 Sep 2026 10:23:57 GMT	Configuración de cabeceras HTTP o cookies
toLocaleDateString()	Formato según la localización del usuario	28/9/2026 (en España: es-ES)	Interfaz de usuario final

4. Métodos de Acceso y Modificación (*Getters* y *Setters*)

Para operar con partes concretas de una fecha se emplean los métodos nativos del objeto:

```
const f = new Date(2026, 8, 28, 14, 45, 10); // 28 de septiembre de 2026

// LECTURA (Getters)
f.getFullYear();      // 2026
f.getMonth();         // 8 (Septiembre, porque Enero es 0)
f.getDate();          // 28 (Día del mes)
```

```
f.getDay();          // Día de la semana (0 = Domingo, 1 = Lunes, ..., 6 = Sábado)
f.getHours();        // 14
f.getMinutes();      // 45
f.getSeconds();      // 10
f.getTime();         // Timestamp en ms (equivalente a valueOf())

// ESCRITURA (Setters)
f.setFullYear(2027);  // Cambia el año a 2027
f.setMonth(0);        // Cambia el mes a enero
f.setDate(15);        // Cambia el día al 15
```

Ejercicios Prácticos

Ejercicio 1: Interpretación de Constructores y Desbordamientos

Analiza el siguiente código sin ejecutarlo en la consola y predice exactamente qué fecha representa cada variable:

```
const fechaA = new Date(2026, 0, 10);
const fechaB = new Date(2026, 12, 1);
const fechaC = new Date(2026);
const fechaD = new Date("2026-02-28");
const fechaE = new Date("2026/02/28");
```

Ejercicio 2: Cálculo de Diferencia Temporal entre Dos Fechas

Implementa una función llamada calcularDiasDiferencia(fechaInicio, fechaFin) que reciba dos cadenas de texto en formato YYYY-MM-DD y devuelva el número entero de días transcurridos entre ambas fechas.

Requisitos:

- Utiliza el cálculo de diferencias en milisegundos mediante .getTime().
- Redondea con Math.round() o trunca con Math.floor() para evitar inconsistencias con cambios horarios (horario de verano/invierno).

Ejercicio 3: Calculadora del Último Día de un Mes

Gracias al comportamiento de desbordamiento, pasar el día 0 al constructor permite obtener el último día del mes inmediatamente anterior.

Crea una función obtenerUltimoDiaMes(año, mes) donde:

- mes se pase en formato humano (1 para enero, 2 para febrero, etc.).
- La función retorna el número entero de días que tiene dicho mes (p. ej., 31, 30, 28 o 29).

Ejercicio 4: Formateador Manual sin Librerías

Escribe una función formatearFechaEspanola(fecha) que reciba un objeto Date y devuelva una cadena con el formato exacto DD/MM/YYYY HH:mm.

Condición: Los números menores de 10 deben incluir un cero a la izquierda (01, 02, ..., 09) utilizando el método de cadenas .padStart(2, "0").

padStart: string.padStart(length, string)

- **length** *(Obligatorio)*: La longitud total que tendrá la cadena resultante después de aplicar el relleno.
- **string** *(Opcional)*: El texto que se usará para rellenar. Si no se especifica, se utiliza un **espacio en blanco** por defecto.

Valor devuelto

- **Tipo**: String (Cadena de texto).
- **Descripción**: Una nueva cadena con la longitud especificada, modificada con el relleno aplicado **desde el principio** (a la izquierda).

Ejemplo padStart:

```
<!DOCTYPE html>

<html>

<body>



<h1>Cadenas de JavaScript</h1>

<h2>El método padStart()</h2>

<p>El método padStart() rellena una cadena desde el principio.</p>

<p>Rellena la cadena con otra cadena (múltiples veces) hasta que alcanza una longitud determinada.</p>

<p id="demo"></p>

<script>

let text = "5";

text = text.padStart(4,"0");

document.getElementById("demo").innerHTML = text;

</script>
```

</body>

</html>

Ejercicio 5: Ejercicio Teórico Comparativo (Ejercicio 9 del libro):

Enunciado: ¿En qué se diferencia técnicamente JavaScript de Java?

Ejercicio 6: Ejercicio de Síntesis de Ventajas (Ejercicio 10 del libro):

Enunciado: Describe las ventajas más importantes de usar JavaScript en el desarrollo web moderno.

Ejercicio 7: Actividad de Laboratorio: Crea un script sencillo en un archivo calculo.js que intente ejecutar una operación matemática con una variable no declarada previamente. Comprueba en el navegador qué sucede: observa cómo las líneas anteriores a la instrucción fallida se ejecutan con normalidad y cómo el intérprete se detiene únicamente al alcanzar el fallo en tiempo de ejecución, comprobando en la consola el error emitido.